Notes:

- HW3, HW4 graded, solutions posted
    - Opened issue on github repo if score on these parts were not 30/30 or 20/20
- HW5 due Today
- HW6 posted
- HW7 posted Thursday
- Final projects coming soon!

Last class:

- Debugging
    - Syntax errors
    - Logical Errors
    - print statement debugging
    - hypothesis testing
    - graphical debugger
    - gdb

Shift our focus to talk about some tools that let us write "better code" for solving scientific computing problems

Many libraries that we could look at:

- Numpy
- Scipy
- pytorch
- TensorFlow
- NetworkX
- SymPy
- Dask

Not enough time to dive into all of these, but we'll make a start

In particular, Numpy is a library that underlies many of these tools (and many others), with data structures commonly using or extending those provided by Numpy

So, what is Numpy?

Primarily, the Numpy library provides an efficient multidimensional array data type. Also provides many useful functions for fast operations on these arrays, such as for mathematical operations, basic linear algebra, and statistics, as well as many more

Basic data type in Numpy is called `ndarray`. Several key differences between this and standard Python data types:

- `ndarray` objects have fixed size at creation, so cannot grow like Python lists do. If you want a bigger array in Numpy, you have to explicitly create a new one.
- All elements of an `ndarray` must be of the same data type.
- Native `ndarray` operations are designed to be executed efficiently on large arrays of data.

There's a catch: we need to learn how to use Numpy's features to write performant code using this data type. Writing code that acts on `ndarray` s but doesn't use native Numpy operations is still expected to be inefficient.

To start, consider:

In [1]:
```python
A_list = [[1, 2, 3], [4, 5, 6]]
```

In [2]:
```python
print(A_list)
```
```
[[1, 2, 3], [4, 5, 6]]
```

In [3]:
```python
type(A_list)
```
Out[3]:
```
list
```

In [4]:
```python
import numpy as np
A_np = np.array(A_list)
print(A_np)
```
```
[[1 2 3]
 [4 5 6]]
```

In [5]:
```python
type(A_np)
```
Out[5]:
```
numpy.ndarray
```

The list form, `A_list`, has no sense of shape, since each element of a list is independent of the others. The Numpy form, `A_np`, has a shape. Has other information as well:

In [6]:
```python
print(A_np.ndim)
```
```
2
```

In [7]:
```python
print(A_np.shape)
```
```
(2, 3)
```

In [8]:
```python
print(A_np.size)
```
```
6
```

In [9]:
```python
print(A_np.dtype)
```
```
int64
```

Notes:

- `ndim` tells us how many dimensions we have
- `shape` gives dimensions
- `size` gives total size
- `dtype` gives data type (here `int64`, meaning 64-bit integers)

Note: Dimensions that we use in `ndarray` s must align:

In [10]:
```python
A2 = np.array([[1,2,3], [4,5]])
```
```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[10], line 1
----> 1 A2 = np.array([[1,2,3], [4,5]])

ValueError: setting an array element with a sequence. The requested array has an inhomogeneous shape after 1 dimensions. The detected shape was (2,) + inhomogeneous part.
```

Doesn't work because it doesn't give even sizes in multidimensional array

Many routines to help us create arrays.

As above, `np.array` converts tuples or lists or lists of lists or lists of tuples into ndarrays

In [11]:
```python
np.array((1,2,3,4))
```
Out[11]:
```
array([1, 2, 3, 4])
```

In [12]:
```python
np.array([(1,2), (3,4), (5,6)])
```
Out[12]:
```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

Can recurse as many times as needed

In [13]:
```python
T = np.array((((1,2), (3,4)), ((5,6), (7,8))))
print(T.shape)
```
```
(2, 2, 2)
```

In [14]:
```python
print(T.ndim)
```
```
3
```

Can also control data type at creation

In [15]:
```python
print(T.dtype)
```
```
int64
```

In [16]:
```python
T = np.array((((1,2), (3,4)), ((5,6), (7,8))), dtype="float")
print(T.dtype)
```
```
float64
```

Can create arrays of all zeros or ones, using

In [17]:
```python
np.zeros((3,7))
```
Out[17]:
```
array([[0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0.]])
```

In [18]:
```python
np.ones((6,2))
```
Out[18]:
```
array([[1., 1.],
       [1., 1.],
       [1., 1.],
       [1., 1.],
       [1., 1.],
       [1., 1.]])
```

In [19]:
```python
print((np.ones((6,2))).dtype)
```
```
float64
```

By default these take type `float64` (standard double precision floating-point numbers)

Can create sequences

In [20]:
```python
np.arange(10)
```
Out[20]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [21]:
```python
np.arange(5,10)
```
Out[21]:
```
array([5, 6, 7, 8, 9])
```

In [22]:
```python
np.arange(5,10,0.5)
```
Out[22]:
```
array([5. , 5.5, 6. , 6.5, 7. , 7.5, 8. , 8.5, 9. , 9.5])
```

Note `arange` returns an ndarray, not the same as the iterator that `range` returns, but you can still use it to control for loop, for example. Also note that you can use floating-point values in `arange`

Alternately, can use `linspace` to specify number of points

In [23]:
```python
np.linspace(1,2,9)
```
Out[23]:
```
array([1.   , 1.125, 1.25 , 1.375, 1.5  , 1.625, 1.75 , 1.875, 2.   ])
```

Note: number of points includes the endpoints, "stop" value is included in the array

If you have an array and want another one of the same dimensions, use `zeros_like` and `ones_like`

In [24]:
```python
np.zeros_like(T)
```
Out[24]:
```
array([[[0., 0.],
        [0., 0.]],

       [[0., 0.],
        [0., 0.]]])
```

Generate random vectors with

In [ ]:
```python
np.random.randint(100, size=(4,2)) # Random integers between 0 and 100
```
Out[ ]:
```
array([[90, 36],
       [46, 61],
       [72,  3],
       [64,  5]])
```

In [ ]:
```python
np.random.random((2,6)) # Uniformly random points between 0 and 1
```
Out[ ]:
```
array([[0.27724993, 0.00399064, 0.096544  , 0.45528437, 0.88324694,
        0.64173238],
       [0.33299094, 0.69753268, 0.73977425, 0.68926254, 0.4316645 ,
        0.09363476]])
```

In [27]:
```python
np.random.normal(size=(6,2)) # Normal distribution, mean zero
```
Out[27]:
```
array([[-1.21732938,  0.39556018],
       [-0.23265509,  1.2563091 ],
       [-0.71525921, -0.40460137],
       [ 0.71839509,  0.18696933],
       [-0.81647235,  1.16674856],
       [-0.39247914, -0.55495454]])
```

All of these have options that control statistics of the numbers generated

In [28]:
```python
help(np.random.normal)
```
```
Help on method normal in module numpy.random:

normal(loc=0.0, scale=1.0, size=None) method of numpy.random.mtrand.RandomState instance
    normal(loc=0.0, scale=1.0, size=None)

    Draw random samples from a normal (Gaussian) distribution.

    The probability density function of the normal distribution, first
    derived by De Moivre and 200 years later by both Gauss and Laplace
    independently [2]_, is often called the bell curve because of
    its characteristic shape (see the example below).

    The normal distributions occurs often in nature.  For example, it
    describes the commonly occurring distribution of samples influenced
    by a large number of tiny, random disturbances, each with its own
    unique distribution [2]_.

    .. note::
        New code should use the `~numpy.random.Generator.normal`
        method of a `~numpy.random.Generator` instance instead;
        please see the :ref:`random-quick-start`.

    Parameters
    ----------
    loc : float or array_like of floats
        Mean ("centre") of the distribution.
    scale : float or array_like of floats
        Standard deviation (spread or "width") of the distribution. Must be
        non-negative.
    size : int or tuple of ints, optional
        Output shape.  If the given shape is, e.g., ``(m, n, k)``, then
        ``m * n * k`` samples are drawn.  If size is ``None`` (default),
        a single value is returned if ``loc`` and ``scale`` are both scalars.
        Otherwise, ``np.broadcast(loc, scale).size`` samples are drawn.

    Returns
    -------
    out : ndarray or scalar
        Drawn samples from the parameterized normal distribution.

    See Also
    --------
    scipy.stats.norm : probability density function, distribution or
        cumulative density function, etc.
    random.Generator.normal: which should be used for new code.

    Notes
    -----
    The probability density for the Gaussian distribution is

    .. math:: p(x) = \frac{1}{\sqrt{ 2 \pi \sigma^2 }}
                     e^{ - \frac{ (x - \mu)^2 } {2 \sigma^2} },

    where :math:`\mu` is the mean and :math:`\sigma` the standard
    deviation. The square of the standard deviation, :math:`\sigma^2`,
    is called the variance.

    The function has its peak at the mean, and its "spread" increases with
    the standard deviation (the function reaches 0.607 times its maximum at
    :math:`x + \sigma` and :math:`x - \sigma` [2]_).  This implies that
    normal is more likely to return samples lying close to the mean, rather
    than those far away.

    References
    ----------
    .. [1] Wikipedia, "Normal distribution",
           https://en.wikipedia.org/wiki/Normal_distribution
    .. [2] P. R. Peebles Jr., "Central Limit Theorem" in "Probability,
           Random Variables and Random Signal Principles", 4th ed., 2001,
           pp. 51, 51, 125.

    Examples
    --------
    Draw samples from the distribution:

    >>> mu, sigma = 0, 0.1 # mean and standard deviation
    >>> s = np.random.normal(mu, sigma, 1000)

    Verify the mean and the standard deviation:

    >>> abs(mu - np.mean(s))
    0.0  # may vary

    >>> abs(sigma - np.std(s, ddof=1))
    0.0  # may vary

    Display the histogram of the samples, along with
    the probability density function:

    >>> import matplotlib.pyplot as plt
    >>> count, bins, ignored = plt.hist(s, 30, density=True)
    >>> plt.plot(bins, 1/(sigma * np.sqrt(2 * np.pi)) *
    ...                np.exp( - (bins - mu)**2 / (2 * sigma**2) ),
    ...          linewidth=2, color='r')
    >>> plt.show()

    Two-by-four array of samples from the normal distribution with
    mean 3 and standard deviation 2.5:

    >>> np.random.normal(3, 2.5, size=(2, 4))
    array([[-4.49401501,  4.00950034, -1.81814867,  7.29718677],   # random
           [ 0.39924804,  4.68456316,  4.99394529,  4.84057254]])  # random
```

When we do arithmetic on arrays, operations are done element-wise

In [29]:
```python
a = np.array([1,2,3,4])
b = np.array([1,-1,2,-2])
a+b
```
Out[29]:
```
array([2, 1, 5, 2])
```

In [30]:
```python
a*b
```
Out[30]:
```
array([ 1, -2,  6, -8])
```

In [31]:
```python
a**2
```
Out[31]:
```
array([ 1,  4,  9, 16])
```

In [32]:
```python
np.cos(b*np.pi)
```
Out[32]:
```
array([-1., -1.,  1.,  1.])
```

These sorts of operations return a new array whose entries are the result of the elementwise operations.

One important thing to realize here is that while we can represent matrices as 2D arrays, `*` does not do regular matrix multiplication

In [34]:
```python
A = np.array([[1, 2], [0, 1]])
B = np.array([[2, 0], [3, 4]])
A*B
```
Out[34]:
```
array([[2, 0],
       [0, 4]])
```

This does a componentwise multiplication. For "regular" matrix multiplication, use `@` or `dot`:

In [35]:
```python
A@B
```
Out[35]:
```
array([[8, 8],
       [3, 4]])
```

In [36]:
```python
np.dot(A,B)
```
Out[36]:
```
array([[8, 8],
       [3, 4]])
```

In [37]:
```python
A.dot(B)
```
Out[37]:
```
array([[8, 8],
       [3, 4]])
```

This also extends to matrix-vector products

In [38]:
```python
x = np.ones((2,1))
A@x
```
Out[38]:
```
array([[3.],
       [1.]])
```

In [39]:
```python
A*x
```
Out[39]:
```
array([[1., 2.],
       [0., 1.]])
```

Note the difference between `x`, which has shape (2,1) and:

In [40]:
```python
y = np.ones(2)
A@y
```
Out[40]:
```
array([3., 1.])
```

Same numerical answer as `A@x`, but one-dimensional object (and printed as a row vector) instead of 2-dimensional

Lots of operations that act on ndarrays:

In [42]:
```python
A = np.random.random((4,6))
A
```
Out[42]:
```
array([[0.56774092, 0.56301098, 0.37257941, 0.10749043, 0.27918419,
        0.02814492],
       [0.86963236, 0.63028108, 0.47809174, 0.95115845, 0.27648272,
        0.56684574],
       [0.4933535 , 0.74185048, 0.51296914, 0.49349497, 0.21161449,
        0.89198793],
       [0.9575163 , 0.74128218, 0.71982823, 0.11610658, 0.88937309,
        0.48939177]])
```

In [43]:
```python
A.min()
```
Out[43]:
```
np.float64(0.02814492074497066)
```

In [44]:
```python
A.max()
```
Out[44]:
```
np.float64(0.9575163009158969)
```

In [45]:
```python
A.sum()
```
Out[45]:
```
np.float64(12.949411614673018)
```

As written here, these operate on entire array. Can also specify an "axis" on which they should operate.

In [46]:
```python
A.min(axis=0)
```
Out[46]:
```
array([0.4933535 , 0.56301098, 0.37257941, 0.10749043, 0.21161449,
       0.02814492])
```

In [47]:
```python
A.sum(axis=1)
```
Out[47]:
```
array([1.91815087, 3.77249208, 3.34527051, 3.91349816])
```

`A.min(axis=0)` means to take the minimum along each column, over index 0

`A.sum(axis=1)` sums over all columns, for each row

Slicing of one-dimensional arrays works just like for lists

In [48]:
```python
a = np.arange(10)**2
print(a)
```
```
[ 0  1  4  9 16 25 36 49 64 81]
```

In [49]:
```python
print(a[1:8:2])
```
```
[ 1  9 25 49]
```

In [50]:
```python
print(a[-1])
```
```
81
```

For multidimensional arrays, can slice on each index:

In [51]:
```python
def f(i,j):
    return 10*i + j**2
b = np.fromfunction(f, (3,7))
print(b)
```
```
[[ 0.  1.  4.  9. 16. 25. 36.]
 [10. 11. 14. 19. 26. 35. 46.]
 [20. 21. 24. 29. 36. 45. 56.]]
```

In [52]:
```python
print(b[1:, ::2])
```
```
[[10. 14. 26. 46.]
 [20. 24. 36. 56.]]
```

Gives rows `1:` and columns `::2` (every other column)

For higher-dimensional ndarrays, use `...` to give all values for a range of indices:

In [53]:
```python
def g(i, j, k, l, m):
    return 10000*i + 1000*j + 100*k + 10*l + m
b = np.fromfunction(g,(2,3,4,5,6))
print(b)
```
```
[[[[[0.0000e+00 1.0000e+00 2.0000e+00 3.0000e+00 4.0000e+00 5.0000e+00]
    [1.0000e+01 1.1000e+01 1.2000e+01 1.3000e+01 1.4000e+01 1.5000e+01]
    [2.0000e+01 2.1000e+01 2.2000e+01 2.3000e+01 2.4000e+01 2.5000e+01]
    [3.0000e+01 3.1000e+01 3.2000e+01 3.3000e+01 3.4000e+01 3.5000e+01]
    [4.0000e+01 4.1000e+01 4.2000e+01 4.3000e+01 4.4000e+01 4.5000e+01]]

   [[1.0000e+02 1.0100e+02 1.0200e+02 1.0300e+02 1.0400e+02 1.0500e+02]
    [1.1000e+02 1.1100e+02 1.1200e+02 1.1300e+02 1.1400e+02 1.1500e+02]
    [1.2000e+02 1.2100e+02 1.2200e+02 1.2300e+02 1.2400e+02 1.2500e+02]
    [1.3000e+02 1.3100e+02 1.3200e+02 1.3300e+02 1.3400e+02 1.3500e+02]
    [1.4000e+02 1.4100e+02 1.4200e+02 1.4300e+02 1.4400e+02 1.4500e+02]]

   [[2.0000e+02 2.0100e+02 2.0200e+02 2.0300e+02 2.0400e+02 2.0500e+02]
    [2.1000e+02 2.1100e+02 2.1200e+02 2.1300e+02 2.1400e+02 2.1500e+02]
    [2.2000e+02 2.2100e+02 2.2200e+02 2.2300e+02 2.2400e+02 2.2500e+02]
    [2.3000e+02 2.3100e+02 2.3200e+02 2.3300e+02 2.3400e+02 2.3500e+02]
    [2.4000e+02 2.4100e+02 2.4200e+02 2.4300e+02 2.4400e+02 2.4500e+02]]

   [[3.0000e+02 3.0100e+02 3.0200e+02 3.0300e+02 3.0400e+02 3.0500e+02]
    [3.1000e+02 3.1100e+02 3.1200e+02 3.1300e+02 3.1400e+02 3.1500e+02]
    [3.2000e+02 3.2100e+02 3.2200e+02 3.2300e+02 3.2400e+02 3.2500e+02]
    [3.3000e+02 3.3100e+02 3.3200e+02 3.3300e+02 3.3400e+02 3.3500e+02]
    [3.4000e+02 3.4100e+02 3.4200e+02 3.4300e+02 3.4400e+02 3.4500e+02]]]


  [[[1.0000e+03 1.0010e+03 1.0020e+03 1.0030e+03 1.0040e+03 1.0050e+03]
    [1.0100e+03 1.0110e+03 1.0120e+03 1.0130e+03 1.0140e+03 1.0150e+03]
    [1.0200e+03 1.0210e+03 1.0220e+03 1.0230e+03 1.0240e+03 1.0250e+03]
    [1.0300e+03 1.0310e+03 1.0320e+03 1.0330e+03 1.0340e+03 1.0350e+03]
    [1.0400e+03 1.0410e+03 1.0420e+03 1.0430e+03 1.0440e+03 1.0450e+03]]

   [[1.1000e+03 1.1010e+03 1.1020e+03 1.1030e+03 1.1040e+03 1.1050e+03]
    [1.1100e+03 1.1110e+03 1.1120e+03 1.1130e+03 1.1140e+03 1.1150e+03]
    [1.1200e+03 1.1210e+03 1.1220e+03 1.1230e+03 1.1240e+03 1.1250e+03]
    [1.1300e+03 1.1310e+03 1.1320e+03 1.1330e+03 1.1340e+03 1.1350e+03]
    [1.1400e+03 1.1410e+03 1.1420e+03 1.1430e+03 1.1440e+03 1.1450e+03]]

   [[1.2000e+03 1.2010e+03 1.2020e+03 1.2030e+03 1.2040e+03 1.2050e+03]
    [1.2100e+03 1.2110e+03 1.2120e+03 1.2130e+03 1.2140e+03 1.2150e+03]
    [1.2200e+03 1.2210e+03 1.2220e+03 1.2230e+03 1.2240e+03 1.2250e+03]
    [1.2300e+03 1.2310e+03 1.2320e+03 1.2330e+03 1.2340e+03 1.2350e+03]
    [1.2400e+03 1.2410e+03 1.2420e+03 1.2430e+03 1.2440e+03 1.2450e+03]]

   [[1.3000e+03 1.3010e+03 1.3020e+03 1.3030e+03 1.3040e+03 1.3050e+03]
    [1.3100e+03 1.3110e+03 1.3120e+03 1.3130e+03 1.3140e+03 1.3150e+03]
    [1.3200e+03 1.3210e+03 1.3220e+03 1.3230e+03 1.3240e+03 1.3250e+03]
    [1.3300e+03 1.3310e+03 1.3320e+03 1.3330e+03 1.3340e+03 1.3350e+03]
    [1.3400e+03 1.3410e+03 1.3420e+03 1.3430e+03 1.3440e+03 1.3450e+03]]]


  [[[2.0000e+03 2.0010e+03 2.0020e+03 2.0030e+03 2.0040e+03 2.0050e+03]
    [2.0100e+03 2.0110e+03 2.0120e+03 2.0130e+03 2.0140e+03 2.0150e+03]
    [2.0200e+03 2.0210e+03 2.0220e+03 2.0230e+03 2.0240e+03 2.0250e+03]
    [2.0300e+03 2.0310e+03 2.0320e+03 2.0330e+03 2.0340e+03 2.0350e+03]
    [2.0400e+03 2.0410e+03 2.0420e+03 2.0430e+03 2.0440e+03 2.0450e+03]]

   [[2.1000e+03 2.1010e+03 2.1020e+03 2.1030e+03 2.1040e+03 2.1050e+03]
    [2.1100e+03 2.1110e+03 2.1120e+03 2.1130e+03 2.1140e+03 2.1150e+03]
    [2.1200e+03 2.1210e+03 2.1220e+03 2.1230e+03 2.1240e+03 2.1250e+03]
    [2.1300e+03 2.1310e+03 2.1320e+03 2.1330e+03 2.1340e+03 2.1350e+03]
    [2.1400e+03 2.1410e+03 2.1420e+03 2.1430e+03 2.1440e+03 2.1450e+03]]

   [[2.2000e+03 2.2010e+03 2.2020e+03 2.2030e+03 2.2040e+03 2.2050e+03]
    [2.2100e+03 2.2110e+03 2.2120e+03 2.2130e+03 2.2140e+03 2.2150e+03]
    [2.2200e+03 2.2210e+03 2.2220e+03 2.2230e+03 2.2240e+03 2.2250e+03]
    [2.2300e+03 2.2310e+03 2.2320e+03 2.2330e+03 2.2340e+03 2.2350e+03]
    [2.2400e+03 2.2410e+03 2.2420e+03 2.2430e+03 2.2440e+03 2.2450e+03]]

   [[2.3000e+03 2.3010e+03 2.3020e+03 2.3030e+03 2.3040e+03 2.3050e+03]
    [2.3100e+03 2.3110e+03 2.3120e+03 2.3130e+03 2.3140e+03 2.3150e+03]
    [2.3200e+03 2.3210e+03 2.3220e+03 2.3230e+03 2.3240e+03 2.3250e+03]
    [2.3300e+03 2.3310e+03 2.3320e+03 2.3330e+03 2.3340e+03 2.3350e+03]
    [2.3400e+03 2.3410e+03 2.3420e+03 2.3430e+03 2.3440e+03 2.3450e+03]]]]



 [[[[1.0000e+04 1.0001e+04 1.0002e+04 1.0003e+04 1.0004e+04 1.0005e+04]
    [1.0010e+04 1.0011e+04 1.0012e+04 1.0013e+04 1.0014e+04 1.0015e+04]
    [1.0020e+04 1.0021e+04 1.0022e+04 1.0023e+04 1.0024e+04 1.0025e+04]
    [1.0030e+04 1.0031e+04 1.0032e+04 1.0033e+04 1.0034e+04 1.0035e+04]
    [1.0040e+04 1.0041e+04 1.0042e+04 1.0043e+04 1.0044e+04 1.0045e+04]]

   [[1.0100e+04 1.0101e+04 1.0102e+04 1.0103e+04 1.0104e+04 1.0105e+04]
    [1.0110e+04 1.0111e+04 1.0112e+04 1.0113e+04 1.0114e+04 1.0115e+04]
    [1.0120e+04 1.0121e+04 1.0122e+04 1.0123e+04 1.0124e+04 1.0125e+04]
    [1.0130e+04 1.0131e+04 1.0132e+04 1.0133e+04 1.0134e+04 1.0135e+04]
    [1.0140e+04 1.0141e+04 1.0142e+04 1.0143e+04 1.0144e+04 1.0145e+04]]

   [[1.0200e+04 1.0201e+04 1.0202e+04 1.0203e+04 1.0204e+04 1.0205e+04]
    [1.0210e+04 1.0211e+04 1.0212e+04 1.0213e+04 1.0214e+04 1.0215e+04]
    [1.0220e+04 1.0221e+04 1.0222e+04 1.0223e+04 1.0224e+04 1.0225e+04]
    [1.0230e+04 1.0231e+04 1.0232e+04 1.0233e+04 1.0234e+04 1.0235e+04]
    [1.0240e+04 1.0241e+04 1.0242e+04 1.0243e+04 1.0244e+04 1.0245e+04]]

   [[1.0300e+04 1.0301e+04 1.0302e+04 1.0303e+04 1.0304e+04 1.0305e+04]
    [1.0310e+04 1.0311e+04 1.0312e+04 1.0313e+04 1.0314e+04 1.0315e+04]
    [1.0320e+04 1.0321e+04 1.0322e+04 1.0323e+04 1.0324e+04 1.0325e+04]
    [1.0330e+04 1.0331e+04 1.0332e+04 1.0333e+04 1.0334e+04 1.0335e+04]
    [1.0340e+04 1.0341e+04 1.0342e+04 1.0343e+04 1.0344e+04 1.0345e+04]]]


  [[[1.1000e+04 1.1001e+04 1.1002e+04 1.1003e+04 1.1004e+04 1.1005e+04]
    [1.1010e+04 1.1011e+04 1.1012e+04 1.1013e+04 1.1014e+04 1.1015e+04]
    [1.1020e+04 1.1021e+04 1.1022e+04 1.1023e+04 1.1024e+04 1.1025e+04]
    [1.1030e+04 1.1031e+04 1.1032e+04 1.1033e+04 1.1034e+04 1.1035e+04]
    [1.1040e+04 1.1041e+04 1.1042e+04 1.1043e+04 1.1044e+04 1.1045e+04]]

   [[1.1100e+04 1.1101e+04 1.1102e+04 1.1103e+04 1.1104e+04 1.1105e+04]
    [1.1110e+04 1.1111e+04 1.1112e+04 1.1113e+04 1.1114e+04 1.1115e+04]
    [1.1120e+04 1.1121e+04 1.1122e+04 1.1123e+04 1.1124e+04 1.1125e+04]
    [1.1130e+04 1.1131e+04 1.1132e+04 1.1133e+04 1.1134e+04 1.1135e+04]
    [1.1140e+04 1.1141e+04 1.1142e+04 1.1143e+04 1.1144e+04 1.1145e+04]]

   [[1.1200e+04 1.1201e+04 1.1202e+04 1.1203e+04 1.1204e+04 1.1205e+04]
    [1.1210e+04 1.1211e+04 1.1212e+04 1.1213e+04 1.1214e+04 1.1215e+04]
    [1.1220e+04 1.1221e+04 1.1222e+04 1.1223e+04 1.1224e+04 1.1225e+04]
    [1.1230e+04 1.1231e+04 1.1232e+04 1.1233e+04 1.1234e+04 1.1235e+04]
    [1.1240e+04 1.1241e+04 1.1242e+04 1.1243e+04 1.1244e+04 1.1245e+04]]

   [[1.1300e+04 1.1301e+04 1.1302e+04 1.1303e+04 1.1304e+04 1.1305e+04]
    [1.1310e+04 1.1311e+04 1.1312e+04 1.1313e+04 1.1314e+04 1.1315e+04]
    [1.1320e+04 1.1321e+04 1.1322e+04 1.1323e+04 1.1324e+04 1.1325e+04]
    [1.1330e+04 1.1331e+04 1.1332e+04 1.1333e+04 1.1334e+04 1.1335e+04]
    [1.1340e+04 1.1341e+04 1.1342e+04 1.1343e+04 1.1344e+04 1.1345e+04]]]


  [[[1.2000e+04 1.2001e+04 1.2002e+04 1.2003e+04 1.2004e+04 1.2005e+04]
    [1.2010e+04 1.2011e+04 1.2012e+04 1.2013e+04 1.2014e+04 1.2015e+04]
    [1.2020e+04 1.2021e+04 1.2022e+04 1.2023e+04 1.2024e+04 1.2025e+04]
    [1.2030e+04 1.2031e+04 1.2032e+04 1.2033e+04 1.2034e+04 1.2035e+04]
    [1.2040e+04 1.2041e+04 1.2042e+04 1.2043e+04 1.2044e+04 1.2045e+04]]

   [[1.2100e+04 1.2101e+04 1.2102e+04 1.2103e+04 1.2104e+04 1.2105e+04]
    [1.2110e+04 1.2111e+04 1.2112e+04 1.2113e+04 1.2114e+04 1.2115e+04]
    [1.2120e+04 1.2121e+04 1.2122e+04 1.2123e+04 1.2124e+04 1.2125e+04]
    [1.2130e+04 1.2131e+04 1.2132e+04 1.2133e+04 1.2134e+04 1.2135e+04]
    [1.2140e+04 1.2141e+04 1.2142e+04 1.2143e+04 1.2144e+04 1.2145e+04]]

   [[1.2200e+04 1.2201e+04 1.2202e+04 1.2203e+04 1.2204e+04 1.2205e+04]
    [1.2210e+04 1.2211e+04 1.2212e+04 1.2213e+04 1.2214e+04 1.2215e+04]
    [1.2220e+04 1.2221e+04 1.2222e+04 1.2223e+04 1.2224e+04 1.2225e+04]
    [1.2230e+04 1.2231e+04 1.2232e+04 1.2233e+04 1.2234e+04 1.2235e+04]
    [1.2240e+04 1.2241e+04 1.2242e+04 1.2243e+04 1.2244e+04 1.2245e+04]]

   [[1.2300e+04 1.2301e+04 1.2302e+04 1.2303e+04 1.2304e+04 1.2305e+04]
    [1.2310e+04 1.2311e+04 1.2312e+04 1.2313e+04 1.2314e+04 1.2315e+04]
    [1.2320e+04 1.2321e+04 1.2322e+04 1.2323e+04 1.2324e+04 1.2325e+04]
    [1.2330e+04 1.2331e+04 1.2332e+04 1.2333e+04 1.2334e+04 1.2335e+04]
    [1.2340e+04 1.2341e+04 1.2342e+04 1.2343e+04 1.2344e+04 1.2345e+04]]]]]
```

In [54]:
```python
print(b[0,1,0,...])
```
```
[[1000. 1001. 1002. 1003. 1004. 1005.]
 [1010. 1011. 1012. 1013. 1014. 1015.]
 [1020. 1021. 1022. 1023. 1024. 1025.]
 [1030. 1031. 1032. 1033. 1034. 1035.]
 [1040. 1041. 1042. 1043. 1044. 1045.]]
```

In [55]:
```python
print(b.ndim)
```
```
5
```

In [56]:
```python
print(b.size)
```
```
720
```

Many ways to manipulate shapes of arrays. `b.flat` is an iterator over the elements in an array:

In [58]:
```python
for element in b[1,0,0,...].flat:
    print(element)
```
```
10000.0
10001.0
10002.0
10003.0
10004.0
10005.0
10010.0
10011.0
10012.0
10013.0
10014.0
10015.0
10020.0
10021.0
10022.0
10023.0
10024.0
10025.0
10030.0
10031.0
10032.0
10033.0
10034.0
10035.0
10040.0
10041.0
10042.0
10043.0
10044.0
10045.0
```

In contrast, `b.flatten()` is a 1-dimensional array that matches the elements of `b`

In [59]:
```python
print(b[1,0,0,...].flatten())
```
```
[10000. 10001. 10002. 10003. 10004. 10005. 10010. 10011. 10012. 10013.
 10014. 10015. 10020. 10021. 10022. 10023. 10024. 10025. 10030. 10031.
 10032. 10033. 10034. 10035. 10040. 10041. 10042. 10043. 10044. 10045.]
```

`b.flatten()` creates a copy of `b` with different dimensions. In contrast, `b.ravel()` creates a "view" of the data in `b`. Changes to `b.ravel()` also change `b`

In [60]:
```python
print(b[0,0,0,0,0])
c = b.ravel()
c[0] = 777
print(b[0,0,0,0,0])
```
```
0.0
777.0
```

In [61]:
```python
d = b.flatten()
d[0] = 8888
print(b[0,0,0,0,0])
```
```
777.0
```

`b.reshape()` is similar to `b.ravel()` but allows more general shaping

In [62]:
```python
a = np.arange(15)
b = a.reshape((3,5))
```

In [63]:
```python
print(a)
b[2,1] = 77
print(b)
```
```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14]
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 77 12 13 14]]
```

In [64]:
```python
print(a)
```
```
[ 0  1  2  3  4  5  6  7  8  9 10 77 12 13 14]
```

Just as with lists, have to be conscious of shallow vs. deep copies

In [65]:
```python
c = a
c[5] = 12
print(a)
```
```
[ 0  1  2  3  4 12  6  7  8  9 10 77 12 13 14]
```

To make a deep copy, use `a.copy()` here:

In [66]:
```python
d = a.copy()
d[5] = -55
print(d)
```
```
[  0   1   2   3   4 -55   6   7   8   9  10  77  12  13  14]
```

In [67]:
```python
print(a)
```
```
[ 0  1  2  3  4 12  6  7  8  9 10 77 12 13 14]
```

When we talk about the "scientific Python ecosystem", we usually identify 3 core libraries:

- Numpy, which provides ndarrays
- Matplotlib, which provides visualization tools
- Scipy, which provides many numerical algorithms

Scipy provides a wide range of tools in a number of submodules. Some of these submodules make sense as standalong packages/modules, while others build heavily on Numpy routines.

A couple of examples of "standalone" submodules that provide useful functionality:

`scipy.constants` provides a number of useful mathematical and physical constants

In [68]:
```python
import scipy.constants as constants
print(constants.pi)
```
```
3.141592653589793
```

In [69]:
```python
print(constants.c) # Speed of light
print(constants.Avogadro) # Avogadro's number
print(constants.ounce) # 1 ounce in kg
print(constants.day) # 1 day in seconds
```
```
299792458.0
6.02214076e+23
0.028349523124999998
86400.0
```

`scipy.special` provides efficient implementations of many special functions. Can call these pointwise or on numpy ndarrays

In [70]:
```python
import scipy.special as special
special.jv(1,1)
```
Out[70]:
```
np.float64(0.44005058574493355)
```

`special.jv(v,z)` evaluates the Bessel Function

$$J_v(z)$$

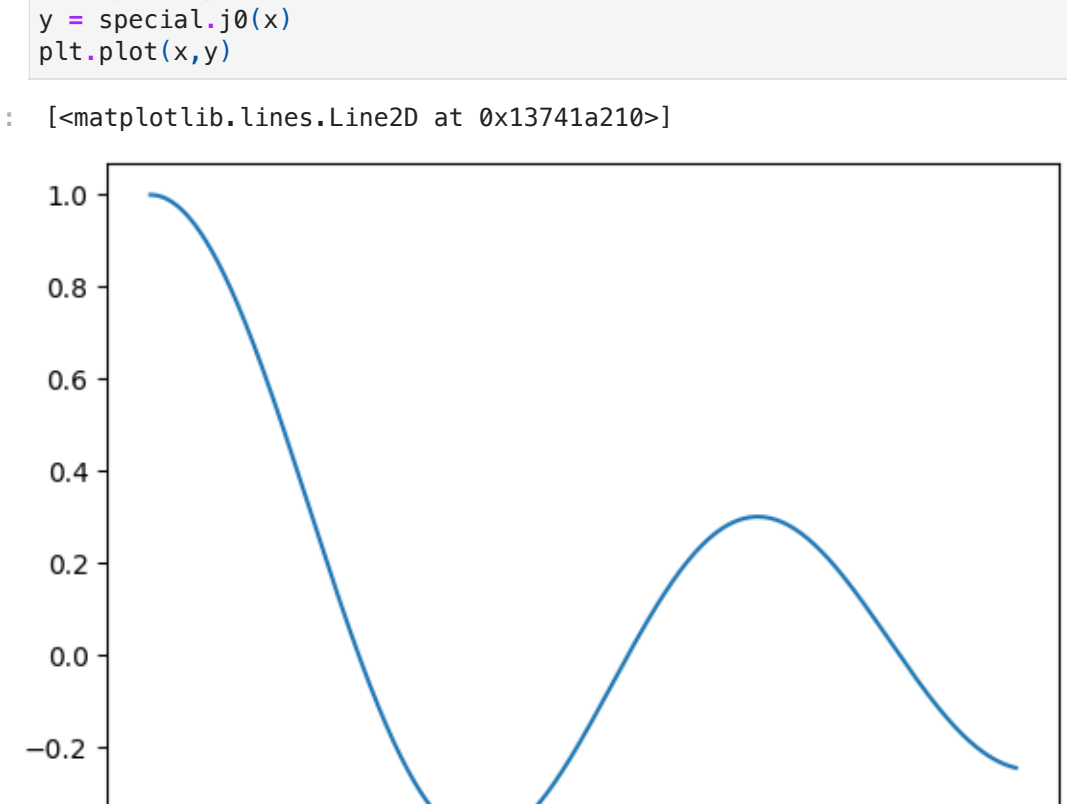
. Bessel functions often arise when working with problems in cylindrical or spherical geometries

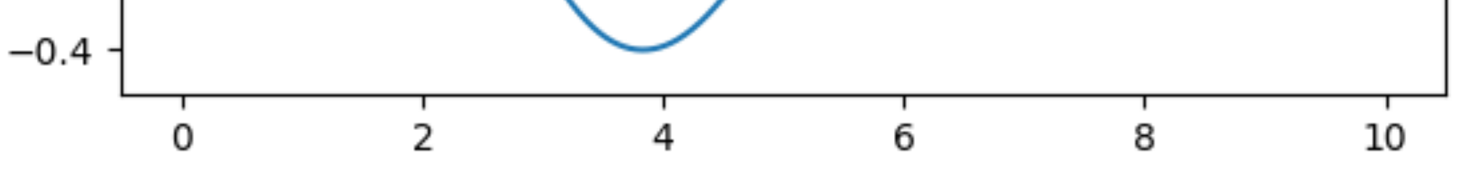
Two faster implementations for Bessel functions of order 0 and 1

In [71]:
```python
special.j0(1)
```
Out[71]:
```
np.float64(0.7651976865579665)
```

In [72]:
```python
special.j1(1)
```
Out[72]:
```
np.float64(0.44005058574493355)
```

In [73]:
```python
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0,10,1000)
y = special.j0(x)
plt.plot(x,y)
```
Out[73]:
```
[<matplotlib.lines.Line2D at 0x13741a210>]
```





Often interested in or have need for zeros of these Bessel functions

In [74]:

```
z = special.jn_zeros(0,3)
print(z)
```

```
[2.40482556 5.52007811 8.65372791]
```

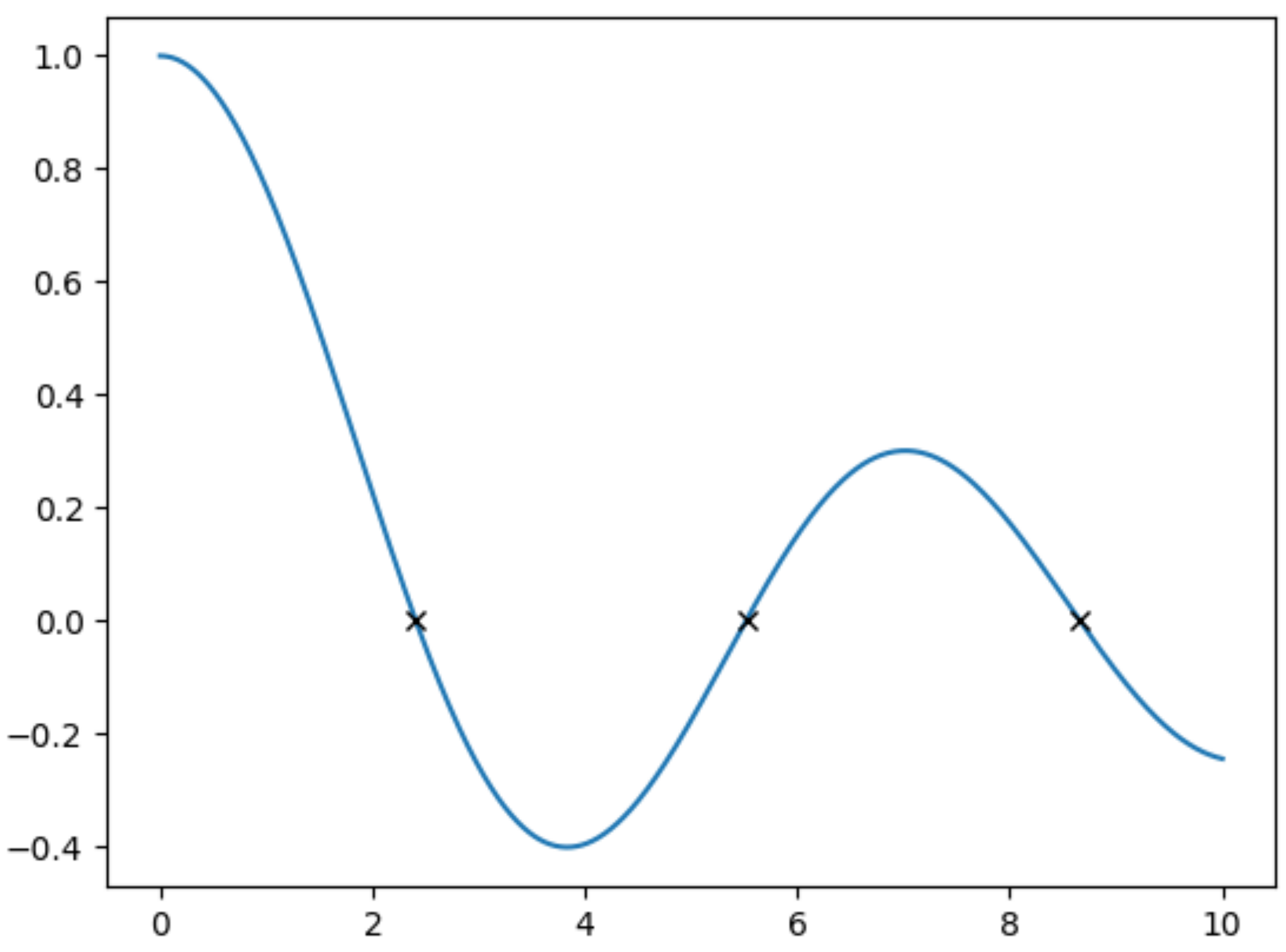
Revisit plot and add these zeros:

In [75]:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0,10,1000)
y = special.j0(x)
plt.plot(x,y)
plt.plot(z,[0,0,0], 'kx ')
```

Out[75]:

```
[<matplotlib.lines.Line2D at 0x1375c7250>]
```



Lots of other common and uncommon special functions available:

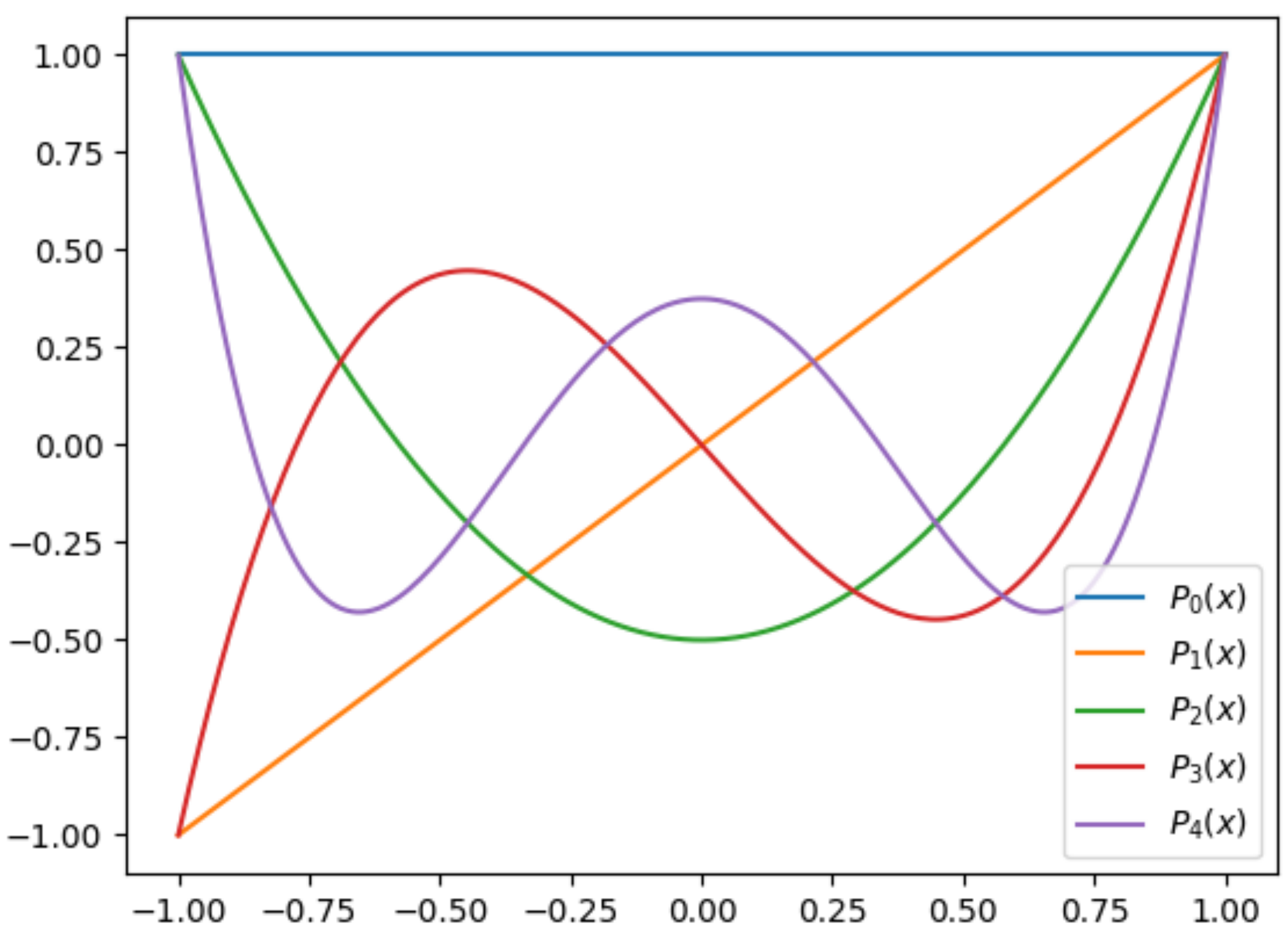
- Error functions, `erf` and `erfc`
- Gamma function, `gamma`
- Orthogonal polynomials:

In [76]:

```
x = np.linspace(-1,1,200)
for n in range(5):
    y = special.eval_legendre(n,x)
    plt.plot(x, y, label=f"$P_{n}(x)$")
plt.legend()
```

Out[76]:

```
<matplotlib.legend.Legend at 0x13748fcb0>
```



Legendre polynomials, orthogonal on [-1,1]

In [79]:

```
for n in range(1,5):
    print(special.roots_legendre(n)[0])
```

```
[0.]
[-0.57735027  0.57735027]
[-0.77459667  0.          0.77459667]
[-0.86113631 -0.33998104  0.33998104  0.86113631]
```

This gives roots of Legendre polynomials of all positive orders. Useful in quadrature (more on this on Thursday)

Also binomial coefficients:

In [80]:

```
special.binom(5,3)
```

Out[80]:

```
np.float64(10.0)
```

In [81]:

```
help(special.binom)
```

```
Help on ufunc:

binom = <ufunc 'binom'>
    binom(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True[, signature])


    binom(x, y, out=None)

    Binomial coefficient considered as a function of two real variables.

    For real arguments, the binomial coefficient is defined as

    .. math::

        \binom{x}{y} = \frac{\Gamma(x + 1)}{\Gamma(y + 1)\Gamma(x - y + 1)} =
            \frac{1}{(x + 1)\mathrm{B}(x - y + 1, y + 1)}

    Where :math:`\Gamma` is the Gamma function (`gamma`) and :math:`\mathrm{B}`
    is the Beta function (`beta`) [1]_.

    Parameters
    ----------
    x, y: array_like
       Real arguments to :math:`\binom{x}{y}`.
    out : ndarray, optional
        Optional output array for the function values

    Returns
    -------
    scalar or ndarray
        Value of binomial coefficient.

    See Also
    --------
    comb : The number of combinations of N things taken k at a time.

    Notes
    -----
    The Gamma function has poles at non-positive integers and tends to either
    positive or negative infinity depending on the direction on the real line
    from which a pole is approached. When considered as a function of two real
    variables, :math:`\binom{x}{y}` is thus undefined when `x` is a negative
    integer.  `binom` returns ``nan`` when ``x`` is a negative integer. This
    is the case even when ``x`` is a negative integer and ``y`` an integer,
    contrary to the usual convention for defining :math:`\binom{n}{k}` when it
    is considered as a function of two integer variables.

    References
    ----------
    .. [1] https://en.wikipedia.org/wiki/Binomial_coefficient

    Examples
    --------
    The following examples illustrate the ways in which `binom` differs from
    the function `comb`.

    >>> from scipy.special import binom, comb

    When ``exact=False`` and ``x`` and ``y`` are both positive, `comb` calls
    `binom` internally.

    >>> x, y = 3, 2
    >>> (binom(x, y), comb(x, y), comb(x, y, exact=True))
    (3.0, 3.0, 3)

    For larger values, `comb` with ``exact=True`` no longer agrees
    with `binom`.

    >>> x, y = 43, 23
    >>> (binom(x, y), comb(x, y), comb(x, y, exact=True))
    (960566918219.9999, 960566918219.9999, 960566918220)

    `binom` returns ``nan`` when ``x`` is a negative integer, but is otherwise
    defined for negative arguments. `comb` returns 0 whenever one of ``x`` or
    ``y`` is negative or ``x`` is less than ``y``.

    >>> x, y = -3, 2
    >>> (binom(x, y), comb(x, y))
    (nan, 0.0)

    >>> x, y = -3.1, 2.2
    >>> (binom(x, y), comb(x, y))
    (18.714147876804432, 0.0)

    >>> x, y = 2.2, 3.1
    >>> (binom(x, y), comb(x, y))
    (0.037399983365134115, 0.0)
```